One Model — tot ce știe un repository

Exerciții: un model, un repository, un view

Pornire

- Proiectul are deja modelul `Product` (cu validări), enum-ul `Category`, `ProductRepository` gol și `ViewProduct` cu repository-ul injectat. Modelul nu se modifică.
- Pornești MySQL cu `docker compose up -d`, apoi aplicația din IntelliJ sau cu `mvn spring-boot:run`.
- Fiecare exercițiu = o metodă privată în `ViewProduct`, chemată din `start()`, în ordinea din fișă.
- În `ProductRepository` scrii doar semnătura metodei. Implementarea o scrie Spring, din numele ei.
- Hibernate tipărește SQL-ul generat (`show-sql: true`): la fiecare metodă nouă citește `where`-ul.
- O listă o afișezi ca nume: `produse.stream().map(Product::getName).toList()`. Unde nu ceri o sortare, ordinea poate varia.
- Fiecare exercițiu se încheie cu un commit `ex N`, pușat.

1 · Populare

Noțiuni noi: `deleteAll` `saveAll` `count`

Faci:

- `populeaza()`: golește tabela cu `deleteAll()`, salvează cele 10 produse cu un singur `saveAll(List.of(...))`, apoi afișează `Produse: ` + `count()`.
- Prețul se scrie `new BigDecimal("9.50")`, data `LocalDate.of(2026, 1, 10)`.

```
name            sku       category   price   stock  active  addedOn
Espresso        COF-0001  COFFEE       9.50    120  true    2026-01-10
Cappuccino      COF-0002  COFFEE      13.00     80  true    2026-01-10
Flat White      COF-0003  COFFEE      14.50      0  true    2026-03-02
Green Tea       TEA-0001  TEA         11.00     40  true    2026-02-15
Black Tea       TEA-0002  TEA         10.00      4  true    2026-02-15
Croissant       PAS-0001  PASTRY       8.00     25  true    2026-04-01
Cheesecake      PAS-0002  PASTRY      18.50      3  true    2026-05-20
Brownie         PAS-0003  PASTRY      12.00      0  true    2026-05-20
French Press    EQP-0001  EQUIPMENT  129.99      7  false   2026-06-11
Coffee Grinder  EQP-0002  EQUIPMENT  249.00      2  true    2026-09-30
```

Gata când:

În consolă apar 10 `insert into products`, apoi `Produse: 10`. La a doua rulare tot `Produse: 10`, nu 20.

2 · Citire

Noțiuni noi: `findAll` `findById` `Optional` `existsById`

Faci:

- Afișezi toate produsele cu `findAll()`, câte unul pe linie (`toString` vine din Lombok).
- Iei `id`-ul primului produs din listă și îl cauți cu `findById`: afișezi `Gasit: ` + numele.
- Cauți id-ul `UUID.fromString("00000000-0000-0000-0000-000000000000")`: afișezi `Exista: ` + `existsById`, apoi `Produs inexistent` dacă `Optional`-ul e gol.

Gata când:

10 linii `Product(id=..., name=..., ...)`, apoi:

```
Gasit: <numele primului produs din listă>
Exista: false
Produs inexistent
```

3 · Validări și sku unic

Noțiuni noi: `saveAndFlush` `ConstraintViolationException` `DataIntegrityViolationException`

Faci:

- Salvezi cu `saveAndFlush` un produs greșit: nume `" "`, sku `abc`, preț `0.00`, stoc `-1`. Prinzi `ConstraintViolationException` (din `jakarta.validation`) și afișezi mesajul fiecărei încălcări din `getConstraintViolations()`, sortate alfabetic.
- Salvezi cu `save` un produs corect, dar cu sku-ul `COF-0001`. Prinzi `DataIntegrityViolationException` și afișezi `Sku already used: COF-0001`.
- La final afișezi din nou `Produse:` + `count()`.

Gata când:

```
Name is required
Price must be at least 0.01
Sku must look like ABC-1234
Stock cannot be negative
Sku already used: COF-0001
Produse: 10
```

La produsul greșit nu apare niciun `insert`: validarea îl oprește înainte de bază. La dublură apare `insert`-ul, respins de MySQL: Hibernate scrie două linii `WARN` cu `Duplicate entry 'COF-0001'`, înaintea mesajului tău.

Încearcă:

- Înlocuiești `saveAndFlush` cu `save`: aplicația se oprește cu `TransactionSystemException`, fiindcă validarea rulează abia la commit. Pui `saveAndFlush` înapoi.

4 · Metode derivate, ghidat

Noțiuni noi: `findBy...` `existsBy...` `countBy...`

Faci:

- Adaugi în `ProductRepository` cele 4 metode și le chemi în view:

```
Optional<Product> findBySku(String sku);
boolean existsBySku(String sku);
List<Product> findByCategory(Category category);
long countByCategory(Category category);
```

Gata când:

```
findBySku("TEA-0001")              ->  Green Tea
findBySku("XXX-0000").isPresent()  ->  false
existsBySku("COF-0002")            ->  true
findByCategory(PASTRY)             ->  Croissant, Cheesecake, Brownie
countByCategory(COFFEE)            ->  3
```

Produsele din `PASTRY` pot ieși în altă ordine.

Încearcă:

- Redenumești metoda în `findBySkuu`: aplicația nu mai pornește, cu `No property 'skuu' found for type 'Product'`. Numele metodei e verificat la pornire, nu la apel.

5 · Metode derivate, singur

Noțiuni noi: cuvintele-cheie din numele metodei

`And`, `Or`	două condiții: `findByCategoryAndStock(...)`
`Between`	două limite, ambele incluse
`LessThan`, `GreaterThan`	comparație strictă; cu `Equal` la final, inclusă
`Before`, `After`	comparație pe date
`Containing`	fragment de text (`like %text%`)
`IgnoreCase`	nu contează litera mare
`True`, `False`	câmp `boolean`, fără parametru
`In`	valoarea e într-o listă primită ca parametru

Faci:

- Câte o metodă derivată pentru fiecare rând, chemată cu valorile din stânga.

Gata când:

Aceleași produse, în orice ordine:

numele conține `TEA`, fără să conteze litera mare	Green Tea, Black Tea
prețul între `10.00` și `13.00`	Cappuccino, Green Tea, Black Tea, Brownie
stocul sub `5`	Flat White, Black Tea, Cheesecake, Brownie, Coffee Grinder
produsele active	9 produse, fără French Press
categoria `EQUIPMENT` și active	Coffee Grinder
categoria e `COFFEE` sau `TEA`, dintr-o listă	Espresso, Cappuccino, Flat White, Green Tea, Black Tea
adăugate după `2026-05-01`	Cheesecake, Brownie, French Press, Coffee Grinder

6 · Sortare, limitare, paginare

Noțiuni noi: `OrderBy` `Top` `Sort` `PageRequest` `Page`

`OrderBy...Asc/Desc`	la finalul numelui: `findByCategoryOrderByPriceDesc`
`Top3`, `First`	după `find`: doar primele rezultate
`Sort`	`findAll(Sort.by("name"))`, `Sort.by(Sort.Direction.DESC, "stock")`
`Pageable`	`findAll(PageRequest.of(pagina, marime, sort))`, paginile încep de la 0
`Page<Product>`	`getContent()`, `getTotalElements()`, `getTotalPages()`

Faci:

- Două metode derivate: cafelele de la cea mai scumpă la cea mai ieftină; cele mai scumpe 3 produse.
- Fără metode noi: toate produsele după nume; toate după stoc, descrescător; pagina a doua din produsele sortate după nume, câte 4 pe pagină, cu `N produse, M pagini`.

- `Page<Product> findByActiveTrue(Pageable pageable)`: prima pagină de 3 produse active, după preț.

Gata când:

```
cafele dupa pret desc    ->  Flat White, Cappuccino, Espresso
top 3 dupa pret          ->  Coffee Grinder, French Press, Cheesecake
dupa nume                ->  Black Tea, Brownie, ..., Green Tea
dupa stoc desc           ->  Espresso, Cappuccino, Green Tea, ...
pagina 2, cate 4         ->  Coffee Grinder, Croissant, Espresso, Flat White
                             10 produse, 3 pagini
active, pagina 1, cate 3 ->  Croissant, Espresso, Black Tea
                             9 produse, 3 pagini
```

La paginare apar două interogări: una cu `limit` și una cu `count`.

7 · @Query

Noțiuni noi: `@Query` `@Param` JPQL `nativeQuery`

- JPQL lucrează cu clasa și câmpurile (`Product p`, `p.addedOn`), SQL-ul nativ cu tabela și coloanele (`products`, `added_on`).

```
@Query("select p from Product p "
     + "where p.stock > 0 and p.price <= :maxPrice order by p.price")
List<Product> findInStockUpTo(@Param("maxPrice") BigDecimal maxPrice);
```

Faci:

- Adaugi metoda de mai sus și o chemi cu `12.00`.
- `BigDecimal stockValue()`, JPQL: suma `price * stock` pe toate produsele.
- `List<Product> findBySkuPrefix(String prefix)`, SQL nativ: sku-ul începe cu `prefix` urmat de `-`, ordonate după sku.

Gata când:

```
findInStockUpTo(12.00)  ->  Croissant, Espresso, Black Tea, Green Tea
stockValue()            ->  4323.43
findBySkuPrefix("PAS")  ->  Croissant, Cheesecake, Brownie
```

8 · Modificare

Noțiuni noi: `save` ca update `@Modifying` `@Transactional`

Faci:

- Iei Espresso după sku, îi pui prețul `10.00` și chemi `save`. Afișezi prețul citit din nou din bază, apoi `Produse: ` + `count()`.
- `int deactivateOutOfStock()`: un `update` JPQL care pune `active = false` pe produsele cu stoc 0. Metoda primește `@Modifying(clearAutomatically = true)` și `@Transactional` (din `org.springframework.transaction.annotation`) și întoarce numărul de rânduri modificate.
- Afișezi `Dezactivate: ` + rezultatul, apoi produsele active.

Gata când:

```
10.00
Produse: 10
Dezactivate: 2
```

Active rămân 7 produse, în orice ordine: lipsesc Flat White, Brownie și French Press.

La `save` apare un `update`, nu un `insert`: produsul are deja `id`.

Încearcă:

- Scoți `@Modifying`: apelul aruncă `InvalidDataAccessApiUsageException`, cu `must be a 'select' query`. Îl pui înapoi.
- Scoți `@Transactional`: aceeași excepție, cu `No active transaction for update or delete query`. Îl pui înapoi.

9 · Ștergere

Noțiuni noi: `deleteById` `deleteBy...`

Faci:

- Iei Black Tea după sku și îl ștergi cu `deleteById`. Afișezi `existsBySku("TEA-0002")`.
- `long deleteByActiveFalse()`, metodă derivată cu `@Transactional`: afișezi `Sterse: ` + rezultatul, apoi `Produse: ` + `count()`.

Gata când:

```
false
Sterse: 3
Produse: 6
```

La `deleteByActiveFalse` apar un `select` și câte un `delete` pe produs: metoda derivată încarcă întâi produsele.

10 · Provocări

Fără indicii. Le chemi în `start()` înainte de exercițiul 8, pe datele inițiale.

cel mai scump produs din `PASTRY`, ca `Optional`, metodă derivată	Cheesecake
câte produse active au stocul sub `5`, metodă derivată	5
produsele adăugate în mai 2026, după nume, metodă derivată	Brownie, Cheesecake
prețul mediu din `TEA`, `@Query` JPQL, întoarce `Double`	10.5